2025 Conference on Building a Secure & Empowered Cyberspace (BuildSEC) | 979-8-3315-7964-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/BUILDSEC68439.2025.00025

An Empirical Analysis of Vulnerability Detection Tools for Ethereum Smart Contracts

Neeraj Kumar
Cybersecurity & Forensics Group (CSFG)
Centre for Development of Advanced Computing (C-DAC)
Patna, India
neeraj12064@gmail.com

Kunal Abhishek
Cybersecurity & Forensics Group (CSFG)
Centre for Development of Advanced Computing (C-DAC)
Patna, India
kunalabh@gmail.com

***Abstract*—Ethereum smart contracts are increasingly being deployed in decentralized finance (DeFi), governance, and automation. However, vulnerabilities in these contracts continue to result in severe security breaches and financial losses. Although there are several automated analysis tools, their effectiveness, efficiency, and scalability in the real world have not been explored enough. We present a large-scale empirical analysis of three state-of-the-art tools, namely Slither, Mythril, and Cyfrin Aderyn leveraging our systematically curated dataset of 16,681 verified Ethereum contracts comprising over 7.2 million lines of Solidity code. We propose a mathematical evaluation framework based on three criteria such as effectiveness, efficiency, and scalability. Our results reveal that Slither offers the broadest coverage and excels in detecting classical vulnerabilities such as reentrancy and access control, whereas Mythril demonstrates deep specialization in arithmetic, randomness, and temporal flaws but suffers from poor scalability due to symbolic execution. Aderyn delivers strong efficiency and emerging threat detection particularly for denial-of-service and unknown unknowns. We found that no single tool ensures comprehensive coverage, yet combining tools yields a balanced defense strategy. Our insights provide actionable guidance for practitioners in tool adoption and aware researchers seeking to design advanced, reliable, and scalable vulnerability detection frameworks.**

***Index Terms*—Smart Contract, Smart Contract Vulnerability, Solidity, Blockchain, Ethereum, Smart Contract Analysis Tools, Testing, Empirical Analysis**

I. Introduction

Blockchain is a decentralized, immutable, digital ledger that records transactions and data across the nodes of the blockchain network. Blockchain technology garnered significant global recognition and accessibility with the publication of Satoshi Nakamoto's groundbreaking work [1] in 2008 which laid the foundational framework for Bitcoin and Ethereum [2]. Today, blockchain technology has expanded its applications well beyond cryptocurrencies. Ethereum introduced a smart contract, a key component of blockchain technology that enables the automation of agreements and transactions between two or more parties.

Ethereum smart contracts are mainly written in Solidity which is a high-level and statically typed programming language. Smart contracts are immutable once they are deployed in the blockchain network. In the realm of decentralized finance (DeFi), vulnerabilities in smart contracts remain a significant concern. The Chainalysis Crypto Report 2024 [3] indicates that exploits targeting smart contracts account for 28.3% of the total value stolen in DeFi hacks annually. The TradeDog 2023 Report [4] documents more than 600 major cyberattacks in the cryptocurrency sector that led to an estimated loss of $2.61 billion, with $291 million attributable to code vulnerabilities. Furthermore, the latest TradeDog Q2 2024 report [5] found that Ethereum as the most targeted blockchain experienced 83 incidents that resulted in $170 million losses of which $40.3 million were due to code-related vulnerabilities.

A. Motivation and Contribution

Many studies [6]–[8] reveal that both novice and experienced developers often introduce vulnerabilities such as reentrancy, access control, arithmetic issues, etc., in smart contracts which expose them to exploitation. Unlike traditional software, contracts cannot be patched after deployment due to blockchain immutability [9], forcing the reliance on upgrade mechanisms that may themselves introduce new vulnerabilities. This immutability magnifies the consequences of security flaws and emphasizes the need for robust vulnerability detection and mitigation in the DeFi ecosystem. Although recent advances have produced a variety of analysis tools [10], [11], recurring exploits demonstrate that existing solutions remain inadequate. This persistent gap highlights the need for systematic and large-scale empirical analysis to rigorously assess current tools and guide the design of more effective, reliable, and scalable solutions.

The main contributions of our paper are as follows.

- We present a systematic empirical analysis based on our novel curated dataset consisting of 16,681 verified Ethereum smart contracts with more than 7.2 million lines of Solidity code, ensuring breadth, diversity, and realism for empirical analysis.
- We design a mathematical evaluation framework based on three criteria such as effectiveness, efficiency, and scalability which are formulated as coverage $E(t)$, contribution C_t, and asymptotic growth $T(N, C)$, respectively in order to capture detection accuracy, category specialization, and tool scalability.
- We conduct a comprehensive study of Slither, Mythril, and Cyfrin Aderyn which uncovers their strengths, lim-

979-8-3315-7964-7/25/$31.00 ©2025 IEEE
DOI 10.1109/BuildSEC68439.2025.00025

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.

itations, and complementarities across the DASP10 vulnerability taxonomy.
- Our multifaceted evaluation provides practical guidance for practitioners on tool adoption and research directions for future analyzers, highlighting that combined deployment strategies yield the most balanced and reliable vulnerability detection.

B. Organization of the Paper

The remainder of this paper is structured as follows. Section II reviews related work on smart contract vulnerabilities and analysis tools highlighting existing gaps. Section III introduces our proposed research approach, detailing the evaluation criteria and tool selection process. Section IV describes our methodology for the empirical analysis including the creation of the data set, the experimental setup, and the categorization of vulnerabilities. Section V reports the results of the analysis according to our defined criteria and provides key insights with practical implications. Finally, Section VI concludes the paper and outlines potential directions for future research.

II. Related Work

Kushwaha et al. [6] reviewed the vulnerabilities of the Ethereum smart contract, notable attacks, and preventive measures but lack empirical evaluations of vulnerability analysis tools. Durieux et al. [12] evaluated nine tools in 69 vulnerable and 47,518 Ethereum contracts that offer insight into automated detection. However, their study is outdated and omits recent tools and analyses across varying contract complexities and sizes, highlighting the need for updated evaluations. Similarly, Ren et al. [10] introduced a unified standard to reduce bias in assessing smart contract testing tools but did not evaluate tools' effectiveness in detecting emerging vulnerabilities or their efficiency in complex contracts. Hwang et al. [13] analyzed 55,046 Ethereum contracts to evaluate the Solidity compiler's ability to address vulnerabilities and the consistency of its updates. Atzei et al. [14] pioneered smart contract security research with a foundational survey on contract attacks that later expanded by Li et al. [15], who introduced a taxonomy categorizing 45 types of vulnerabilities. Chen et al. [16] examined vulnerabilities, attacks, and defenses, providing information on root causes, attack impacts, and defensive strategies. Jiao et al. [7] classified vulnerabilities using a three-tier threat model and evaluated 14 analysis tools while Khan et al. [17] reviewed 41 tools categorizing their detection techniques and emphasizing innovation needs. Recently, Chen et al. [11] evaluated the performance of ChatGPT in detecting vulnerabilities using a public dataset to evaluate its precision and effectiveness. Recent studies [18], [19] have introduced AI and ML assisted approaches for smart contract vulnerability detection. These methods leverage deep learning, code embeddings, and large language models (LLMs) to infer vulnerability patterns without predefined rules. Graph neural networks and transformer-based models capture semantic and contextual code dependencies improving generalization and adaptive detection. However, they also face challenges in explainability, reproducibility, and dataset bias.

Overall, existing analyses on smart contract vulnerability detection provide valuable foundations but exhibit notable limitations. Previous works either focus on surveys without empirical validation, rely on outdated tool sets, or fail to account for logical complexities and the scalability challenges posed by large and real-world contracts. Given the rapid evolution of the DeFi ecosystem, there is a clear need for updated evaluations. Our analysis addresses this gap through a large-scale empirical analysis grounded in mathematical rigor that offers timely insights to guide the development of more effective and scalable vulnerability detection tools.

III. The Proposed Research Approach

We present the approach of our analysis through the formulation of evaluation criteria and a systematic process for tool selection. The objective is to establish a rigorous and transparent mathematical framework for assessing Solidity smart contract analysis tools.

A. The Proposed Evaluation Criteria

Existing studies often evaluate tools only in terms of accuracy or case-specific performance overlooking critical aspects like computational cost and scalability. With contracts becoming increasingly complex and deployed in massive volumes such limited evaluations fail to reflect real-world challenges. To address this gap, we proposed three new criteria, that is, effectiveness, efficiency, and scalability, formalized through mathematical representations and enabling a holistic assessment of vulnerability detection tools. Table I presents the evaluation criteria used to compare the smart contract analysis tools.

- Criteria#1. [Tool Effectiveness] The effectiveness of an analysis tool is measured in terms of its ability to accurately detect vulnerabilities in Solidity smart contracts. Let $\mathcal{V} = \{v_1, v_2, \ldots, v_K\}$ denote the set of vulnerability categories, where $K = 10$ corresponds to the DASP10 [20] classification. Let $\mathcal{T} = \{\text{Slither}, \text{Mythril}, \text{Aderyn}\}$ represent the set of analysis tools.

  We define a binary detection matrix as shown in Equation 1:

$$M(t,v) = \begin{cases} 1 & \text{if tool } t \in T \text{ detects category } v \in V, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

  The effectiveness score of a tool t is given by its coverage ratio as defined in Equation 2:

$$E(t) = \frac{\sum_{v \in V} M(t,v)}{K} \tag{2}$$

  which measures the proportion of vulnerability categories successfully detected by tool t.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.

To account for the varying degree of contribution across categories (as shown in Figure 1), we further define the normalized contribution vector for tool in Equation 3:

$$C_t = (ct_1, ct_2, \ldots, ct_K), \quad \sum_{v=1}^{K} ct_v = 1 \tag{3}$$

where c_{tv} represents the proportion of detections contributed by tool t in category v relative to all tools combined.
Thus, tool effectiveness can be characterized as shown in Equation 4:

$$Effectiveness(t) = \big(E(t), C_t\big) \tag{4}$$

where $E(t)$ captures categorical coverage, and $\mathbf{C}_t$ highlights specialization or dominance in specific vulnerability classes.
These metrics provide insight into how well tools such as Mythril, Slither, and Cyfrin Aderyn identify both known and emerging vulnerabilities in Ethereum smart contracts.

- Criteria#2. [Tool Efficiency] Efficiency refers to the computational cost required to analyze a smart contract. Formally, let $T(n)$ denote the time complexity of the analysis as a function of the contract size n (measured in lines of code, functions, or bytecode length). We empirically record the runtime τ for each tool when analyzing contracts of varying sizes. Efficiency is evaluated by comparing average runtime as expressed in Equation 5:

$$\text{Avg. Runtime} = \frac{1}{m}\sum_{i=1}^{m} \tau_i \tag{5}$$

  where m is the number of contracts analyzed. This allows us to determine which tool provides the best trade-off between detection accuracy and runtime performance.

- Criteria#3. [Tool Scalability] Scalability assesses how well the tools handle an increasing number of contracts and growing contract complexity. If C represents the complexity of a contract (e.g., number of functions, modifiers, inheritance depth, cyclomatic complexity), and N denotes the number of contracts, then scalability can be analyzed through asymptotic behavior of $T(N, C)$. A scalable tool should exhibit near-linear or sub-quadratic growth with respect to both N and C. We experimentally evaluate whether performance degrades significantly for large-size contracts and identify tools capable of maintaining reliable analysis under high-load scenarios.

B. Tool Selection

We surveyed prominent tools for analyzing vulnerabilities in solidity smart contracts in [12], [21], [22]. Based on this review, we identified seven widely recognized tools as summarized in Table II. Certain tools were excluded from our study due to specific limitations and constraints, and the rationale for their exclusion is discussed in the following section.

Several tools were excluded from our study due to specific limitations. First, Oyente was found to be incompatible with recent Solidity versions as it only supports version 0.4.19 and does not analyze contracts above version 0.5.0 [30]. Attempts to use newer versions (e.g., 0.5.x and above) often result in errors or warnings underscoring the urgent need for updates to align with evolving Solidity standards. Furthermore, tools derived from Oyente such as Osiris and Honeybadger were excluded since they inherit the same architectural constraints and lack advanced functionalities required for comprehensive vulnerability detection. Lastly, Maian was excluded because of its limited capability to analyze complex smart contracts which makes it unsuitable for large-scale and diverse datasets.

Following a comprehensive evaluation of seven tools, we selected three standout tools for our empirical analysis: Slither, Mythril, and Cyfrin Aderyn. Slither [23] developed by Trail of Bits in 2017 is a Python-based static analysis framework that translates Solidity code into an intermediate representation (SlithIR) and applies data flow and taint analysis for vulnerability detection. Mythril [24], which is part of ConsenSys' MythX suite (2018), employs symbolic execution to identify vulnerabilities in contracts on EVM compatible blockchains. Cyfrin Aderyn [25] introduced in 2024 by Cyfrin Labs is a Rust-based AST analyzer that leverages specialized detectors to uncover vulnerability patterns in Solidity smart contracts. These tools were selected for their robustness, reliability, and ability to meet critical requirements for effective smart contract analysis.

IV. Methodology of Empirical Analysis

We describe the systematic methodology adopted in our empirical analysis. We first outline the process of creating the dataset followed by the experimental setup and analysis methodologies. Finally, we detail the vulnerability categorization and the evaluation framework which emphasize the three core criteria i.e., effectiveness, efficiency, and scalability of the selected tools.

A. Dataset Creation

We built a comprehensive and diverse dataset of Ethereum smart contracts where we collected metadata on $N = 16,681$ verified contracts from Etherscan [31], which is a widely recognized blockchain explorer for the Ethereum network. The metadata retrieved in CSV format included essential attributes such as transaction hashes (tx_i), contract addresses ($addr_i$), contract names, compiler versions, and creation timestamps formally represented as shown in Equation 6.

$$\mathcal{M} = \{(tx_i, addr_i, name_i, comp_i, time_i)\}_{i=1}^{N}. \tag{6}$$

Using these metadata $\mathcal{M}$, we programmatically retrieved both the source code and the corresponding EVM bytecode for each contract. This resulted in a dataset as provided in Equation 7.

$$D = \{(C_i^{src}, C_i^{bin}) \mid i = 1, 2, \ldots, N\} \tag{7}$$

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EVALUATION CRITERIA FOR SMART CONTRACT ANALYSIS TOOLS

Criterion	Definition	Mathematical Metric(s)
Effectiveness	Accuracy in detecting known and emerging vulnerabilities	$E(t) = \frac{\sum_{v \in \mathcal{V}} M(t,v)}{K}$, $\mathbf{C}_t = (c_{t1}, c_{t2}, \ldots, c_{tK}), \quad \sum_{v=1}^{K} c_{tv} = 1,$ Effectiveness$(t) = \big(E(t), \mathbf{C}_t\big)$
Efficiency	Time required for analysis as a function of contract size n	Avg. Runtime $= \frac{1}{m}\sum_{i=1}^{m} \tau_i, \quad T(n)$
Scalability	Ability to handle large datasets (N contracts) and complex contracts (C)	$T(N, C)$ (growth should be near-linear or sub-quadratic)

TABLE II
POTENTIAL CANDIDATE TOOLS FOR SOLIDITY VULNERABILITY ANALYSIS

Tool	URL
Slither [23]	https://github.com/crytic/slither
Mythril [24]	https://github.com/Consensys/mythril
Cyfrin Aderyn [25]	https://github.com/Cyfrin/aderyn
Maian [26]	https://github.com/ivicanikolicsg/MAIAN
Oyente [27]	https://github.com/enzymefinance/oyente
Honeybadger [28]	https://github.com/christoftorres/HoneyBadger
Osiris [29]	https://github.com/danielkrupinski/Osiris

where C_i^{src} denotes the Solidity source code and C_i^{bin} denotes the compiled bytecode of the verified contract i^{th}.
In total, $\mathcal{D}$ comprised $7,213,557$ lines of code (LOC), spanning contracts of varying complexity and sizes. The LOC distribution per contract follows a right-skewed pattern, with a median of $\tilde{L}$ and a maximum exceeding L_{max} lines, illustrating the diversity of contract sizes and coding practices in the Ethereum ecosystem.
To ensure high-quality data, we performed the following preprocessing steps:

- Deduplication: Contracts with identical bytecode but different deployment addresses were merged to avoid redundancy.
- Parsing and Normalization: The source files were normalized by removing comments and formatting artifacts, ensuring consistent tokenization for downstream analysis.
- Feature extraction: For each contract, we extracted structural features including the number of functions, modifiers, events, and inheritance depth. At the bytecode level, opcode frequency vectors were generated as shown in Equation 8.

$$f_i = (f_{i1}, f_{i2}, \ldots, f_{im}) \tag{8}$$

where f_{ij} represents the frequency of the opcode j in contract i, and m denotes the total number of unique EVM opcodes.

For a comprehensive comparative analysis, the raw output was converted into a standardized `.CSV` format. Each `.CSV` file captured essential attributes such as contract address, start time of analysis, end time of analysis, detected vulnerabilities, and lines of code (LOC). This dataset thus provides a multi-view representation of Ethereum smart contracts, spanning lexical (LOC), syntactic (AST), and semantic (CFG/opcode) levels. Inclusion of both source code and bytecode ensures compatibility with static analysis, dynamic analysis, and machine learning frameworks. By covering contracts of diverse sizes, complexity, and functionality, the dataset offers a representative and practical benchmark for empirical studies on smart contracts on Ethereum.

The dataset was also categorized based on the Solidity pragma versions specified in the smart contract source code. As shown in Table III, most contracts (approximately 98.66%) target Solidity version 0.8.x and above which indicates a clear shift toward adopting the latest stable compiler releases. This trend reflects developers' preference for enhanced security, better compiler efficiency, and the advanced features offered by newer Solidity versions. Although our dataset predominantly includes Solidity 0.8.x contracts, this reflects the real-world shift toward newer versions. However, future extensions will include legacy contracts (0.4.x–0.6.x) to ensure historical vulnerability coverage. The curated dataset and analysis scripts are available upon request for research verification.

TABLE III
SOLIDITY PRAGMA VERSION COUNT

Pragma Version	# of Contracts
0.4.x	55
0.5.x	14
0.6.x	73
0.7.x	90
0.8.x	16449

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.

B. Experimental Set-up

We systematically selected, configured, and executed the three state-of-the-art analysis tools selected within an isolated environment to ensure uniform testing conditions. All tools were run with identical hardware specifications consisting of 16 GB RAM, 8 CPU cores, and 1 TB storage. This standardized set-up was adopted to guarantee consistency, fairness, and comparability in the empirical analysis.

- Mythril employs symbolic execution, control flow analysis, and taint analysis to identify vulnerabilities in the bytecode of smart contracts. Mythril's significant computational demands highlighted scalability challenges for broader applications.
- For Slither, the results were saved in JSON format, capturing both vulnerabilities and associated detectors. Slither's lightweight design allowed for efficient processing of contracts which makes it suitable for large scale analysis on moderately resourced systems.
- Cyfrin Aderyn generates structured reports in Markdown (.md) format summarizing vulnerabilities and their classifications.

C. Vulnerability Categorization

We evaluated the effectiveness of tools in identifying vulnerabilities by systematically categorizing detected issues using the DASP10 taxonomy [20], a widely recognized framework for classifying vulnerabilities in smart contracts. The categorization process involved the following steps:

- Mapping of the vulnerability descriptions identified by Mythril to their corresponding categories within the DASP10 taxonomy.
- Classifying the outputs of Slither and Aderyn on the basis of their predefined detection mechanisms and the associated types of vulnerability.

This approach ensured a standardized comparison of the tools' capabilities in detecting and categorizing vulnerabilities across multiple dimensions.

D. Discussion on Performance Metrics

It is important to emphasize that the three evaluation criteria that is effectiveness, efficiency, and scalability also serve as the performance metrics for comparing tools in our empirical analysis. Specifically, effectiveness captures detection accuracy and category coverage, efficiency reflects computational cost through average runtime, and scalability assesses the ability to handle large datasets and complex contracts. Together, these metrics provide a balanced and comprehensive basis for tool comparison which address both technical precision and real-world applicability.

V. Results & Analysis

We present the results of the empirical analysis conducted using our dataset, the proposed evaluation criteria, and the experimental setup as described in the earlier sections. We systematically analyze the performance of the selected tools with respect to our proposed criteria such as effectiveness, efficiency, and scalability, providing both quantitative results and qualitative insight into their strengths and limitations.

TABLE IV
Tool Effectiveness in Detecting DASP10 Vulnerabilities

DASP10 Category	Slither	Mythril	Aderyn
Access Control	✓	✓	✓
Arithmetic	✓	✓	✓
Unchecked Low Level Calls	✓	✓	✓
Denial of Service	✓	×	✓
Bad Randomness	×	✓	✓
Time Manipulation	✓	✓	×
Reentrancy	✓	×	×
Unknown Unknowns	✓	✓	✓
Front-Running	×	×	×
Short Address Attack	×	×	×

A. Criteria #1. [Tool Effectiveness]

We aim to assess the precision of the analysis tools such as Mythril, Slither and Cyfrin Aderyn in the identification of not only known vulnerabilities but also new and emerging vulnerabilities in Etherum smart contracts. Figure 1 presents a normalized contribution (in percentage) of each tool in different vulnerability categories. Table IV indicates the effectiveness of the tool in detecting DASP10 vulnerabilities.

.

- Access Control: Slither emerges as the dominant tool for access control vulnerabilities which include improper authorization and privilege escalation. Although not achieving full dominance, Slither shows strong coverage, complemented by Cyfrin Aderyn, which provides notable secondary contributions. This indicates Slither's prioritization of foundational security flaws while Aderyn effectively supports detection of permission-based weaknesses. In contrast, Mythril contributes minimally in this category which suggests that access control is not its primary strength.
- Reentrancy: Slither stands out as the most effective tool in detecting reentrancy vulnerabilities that allow recursive calls to drain contract funds. Neither Mythril nor Aderyn detects this category, indicating their limited effectiveness for classical high-impact flaws. These results are consistent with previous findings [30] showing Mythril's challenges in detecting reentry issues in Solidity 0.8.x and above. Our observations also align with [32] confirming Mythril's difficulties in analyzing data flow during cross-contract interactions, leading to missed detections.
- Denial of Service (DoS): In DoS vulnerabilities, Cyfrin Aderyn takes the lead, which focuses on threats related to resource management and contract availability. Slither provides moderate support showcasing its broader but less specialized coverage. However, Mythril does not contribute significantly in this category.
- Arithmetic and Low-Level Calls: Mythril demonstrates clear dominance in arithmetic vulnerabilities including overflows and underflows which are critical in financial

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.

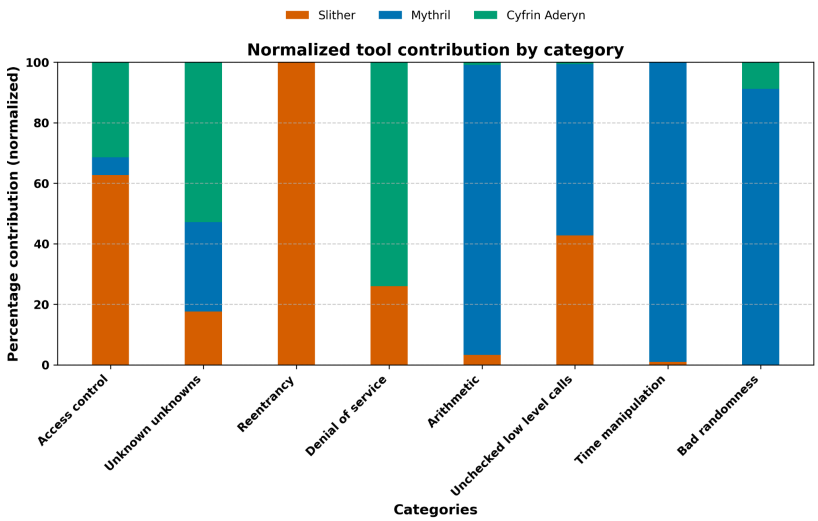

Fig. 1. Normalized Contribution of Each Tool across Vulnerability Categories

contracts. It also leads to the detection of unchecked low-level calls, which highlights its effectiveness in identifying flaws at the EVM interaction layer. Slither offers only minor support in these categories, while Aderyn shows no contribution, likely due to its emphasis on other vulnerability classes.

- Time Manipulation and Randomness: Time manipulation vulnerabilities involving exploitable timestamp dependencies are almost exclusively detected by Mythril underscoring its specialization in temporal flaws. Similarly, Mythril is the only tool that provides meaningful detection of bad randomness vulnerabilities which are common in lottery and gaming applications. Slither and Aderyn show negligible contributions in both categories.
- Unknown Unknowns: For emerging and less-defined vulnerabilities, Cyfrin Aderyn clearly dominates. Its superior detection capability highlights its strength in capturing novel and evolving threats. Mythril provides moderate secondary coverage, whereas Slither contributes minimally. This reflects Aderyn's forward-looking design, which makes it particularly valuable for identifying vulnerabilities that are not yet well documented or categorized.

We also quantify the effectiveness of the tool in terms of category coverage $E(t)$ and dominance/specialization C_t. Coverage is measured as the fraction of DASP10 categories where a tool provides non-zero contribution as expressed in Equation 9.

$$E(t) = \frac{\#\,\text{categories detected by tool } t}{10} \tag{9}$$

Dominance is assessed by normalized contribution per category (Figure 1). A tool is considered dominant in a category if its share is $\geq 70\%$ of the normalized contribution.

Our analysis suggests that the coverage-based evaluation using $E(t)$ and C_t demonstrates that no individual tool achieves complete effectiveness across the DASP10 categories. Slither attains the highest breadth of coverage with $E(\text{Slither}) = 0.7$, driven by strong category contributions C_t in Reentrancy and Access Control. Mythril follows with $E(\text{Mythril}) = 0.6$, where its C_t values show clear dominance in Arithmetic, Time Manipulation, and Bad Randomness reflecting deep specialization in computational and temporal vulnerabilities. Aderyn also produces $E(\text{Aderyn}) = 0.6$ with distinctive contributions C_t in unknown unknowns and denial of service, indicating strength in emerging and less conventional vulnerability classes. These findings confirm that the tools exhibit complementary profiles: Slither ensures robustness against classical high-impact flaws, Mythril enhances detection of financial and randomness-related risks, and Aderyn expands the assurance into evolving threat categories. Consequently, a combined deployment strategy achieves the most balanced and comprehensive security coverage for Ethereum smart contracts.

B. Criteria #2. [Tool Efficiency]

We evaluated and compared the performance of these analysis tools to determine which is the most efficient. Table V provides a detailed comparison of the efficiency of three selected smart contract analysis tools: Slither, Mythril, and Cyfrin Aderyn evaluated over a dataset of 16,681 smart contracts. Slither stands out as the most efficient and scalable tool, achieving an average execution time of just 0.9 seconds per contract with a total analysis duration of 4 hours and 48 minutes. Its static analysis approach prioritizes speed which makes it highly suitable for real-time or large-scale audits where rapid contract processing is essential. Cyfrin Aderyn with an average execution time of 1.6 seconds per contract

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.

and a total runtime of 7 hours and 24 minutes balances speed and analytical depth positioning itself as a versatile tool for scenarios requiring moderately detailed analysis without sacrificing scalability.

TABLE V
EXECUTION TIME FOR EACH TOOL

Tools	Average Time	Total Time
Slither	0 days 00:00:00.901102902	0 days 04:48:41
Mythril	0 days 00:16:16.49	7 days 04:51:59
Cyfrin Aderyn	0 days 00:00:01.598225526	0 days 07:24:20

On the other hand, Mythril leveraging symbolic execution and deeper exploration recorded an average execution time of 16 minutes and 16 seconds per contract resulting in a total analysis duration exceeding 7 days. Although this significantly longer runtime reflects its ability to uncover complex vulnerabilities and perform comprehensive audits, it highlights its limited scalability for large datasets. These results reveal a clear trade-off between speed and depth: Slither excels in scenarios demanding high scalability and efficiency, Mythril is tailored for in-depth detailed analysis at the cost of runtime, and Cyfrin Aderyn provides a middle ground making it a practical choice for balanced performance. This evaluation underscores the importance of selecting an analysis tool based on specific requirements such as dataset size, contract complexity, and the need for either rapid or exhaustive vulnerability detection.

C. Criteria #3. [Tool Scalability]

We evaluated tool scalability by analyzing how execution time $T(N, C)$ varies with the number of contracts (N) and the complexity of the contract (C). A tool is considered scalable when $T(N, C)$ exhibits near-linear or sub-quadratic growth as both N and C increase.

For this purpose, we used a dataset of 16,681 contracts comprising 7,213,557 lines of code (LOC). Contracts were grouped into three categories based on LOC: Small (100–1000 LOC), Medium (1001–5000 LOC), and Large (5001+ LOC). Table VI reports the average elapsed time (in seconds) for Slither, Mythril, and Cyfrin Aderyn in these categories.

TABLE VI
AVERAGE ANALYSIS TIME (IN SECONDS) PER TOOL ACROSS CONTRACT SIZES

Tool	Small	Medium	Large
Slither	0.967	1.150	13.538
Mythril	1052.222	581.034	2340.769
Aderyn	1.589	1.742	2.923

Our findings reveal clear asymptotic differences in scalability among the tools:

- Slither: Exhibits near-linear scalability with $T(N, C)$ increasing modestly from 0.967s (Small) to 13.538s (Large). This growth pattern indicates sub-quadratic behavior which makes Slither the most scalable and efficient tool for large-scale repositories and time-sensitive analyses.
- Aderyn: Maintain consistently low execution times across categories (1.589 s $\rightarrow$ 2.923 s), showing a very weak dependence on C and demonstrating stable scalability. Although slightly slower than Slither, Aderyn offers a strong trade-off between speed and coverage.
- Mythril: Shows non-scalable behavior with $T(N, C)$ sharply increasing as contracts grow in size (1052.222s $\rightarrow$ 2340.769s). The super-linear growth highlights the computational overhead of symbolic execution which makes Mythril unsuitable for large-scale systems but effective for small contracts requiring deep vulnerability exploration.

Additionally, we observe a practical limitation: Mythril's bytecode analysis required manual specification of the compiler version (e.g., 0.6.12), while Slither and Aderyn automatically inferred it from pragma directives in the source code. This reinforces the scalability advantage of source code-based tools when handling diverse contracts and compiler versions. We also found that Slither and Aderyn achieve near-linear scalability in $T(N, C)$, which makes them well suited for large-scale contract analysis. In contrast, Mythril's super-linear behavior and high execution costs restrict its practicality to smaller-scale or highly specialized analyses where precision outweighs performance. Beyond computational scalability, practical adoption of smart contract analysis tools also depends on memory usage, developer usability, and CI/CD integration. Our observations indicate that Slither and Aderyn exhibit low memory footprints and are easier to integrate into automated testing workflows, whereas Mythril's symbolic execution requires substantial memory and manual configuration which limits its suitability for large-scale or real-time environments.

Although our study primarily focused on quantitative evaluation metrics such as coverage, runtime, and scalability, but we acknowledge that false-positive and false-negative analyses provide complementary insight into real-world reliability. The observed detection patterns where tools like Slither exhibit broader coverage and Mythril offers deeper but more selective analysis already reflect implicit trade-offs between precision and recall. However, establishing a validated ground truth across large-scale datasets remains challenging.

VI. CONCLUSION

We conducted a large-scale empirical analysis of vulnerability detection tools for Ethereum smart contracts using our systematically curated dataset of 16,681 verified contracts comprising over 7.2 million lines of code. Our study introduced a mathematical framework grounded in three evaluation criteria such as effectiveness, efficiency, and scalability. The results reveal distinct trade-offs with our proposed criteria. Slither provides the broadest coverage ($E = 0.7$), excelling in access control and reentrancy but underperforming in bad

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.

randomness, time manipulation, and unchecked low-level calls reflecting its limits in computational and temporal vulnerabilities. Mythril demonstrates deep specialization with C_t dominance in arithmetic, randomness, and temporal flaws but suffers from scalability limitations due to symbolic execution. Cyfrin Aderyn offers high efficiency and strong performance in emerging categories such as denial-of-service and unknown unknowns highlighting its forward-looking capability. Overall, no single tool achieves full-spectrum detection; however, complementary deployment (e.g., Slither + Mythril or Mythril + Aderyn) ensures broader coverage. These findings serve as an important reminder to practitioners who may use vulnerability analysis tools without fully considering the critical criteria required for an effective analyzer. Our results provide actionable insights for practitioners in selecting tools that are best suited to their needs, while also offering researchers clear directions for developing next-generation analyzers that balance effectiveness, efficiency, and scalability.

VII. ACKNOWLEDGMENT

We gratefully acknowledge the Centre for Development of Advanced Computing (C-DAC) for its support and guidance. We sincerely thank Mr. Abhinav Dixit, Centre Head, and Mr. Aditya Kumar Sinha, Former Centre Head, C-DAC Patna for their invaluable encouragement and guidance. We also thank the reviewers for their insightful feedback that helped refine this paper.

REFERENCES

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[2] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[3] C. Team. (2024) The chainalysis 2024 crypto crime report. [Online]. Available: https://www.chainalysis.com/blog/crypto-hacking-stolen-funds-2024/

[4] (2023) A comprehensive analysis of crypto hacks and exploits in 2023. [Online]. Available: https://tradedog.io/a-comprehensive-analysis-of-crypto-hacks-and-exploits-in-2023/

[5] (2024) Crypto hacks rise to 688 million dollars in q2 2024. [Online]. Available: https://tradedog.io/crypto-hacks-rise-to-688-mn-in-q2/

[6] S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, "Systematic review of security vulnerabilities in ethereum blockchain smart contract," *IEEE Access*, vol. 10, pp. 6605–6621, 2022.

[7] T. Jiao, Z. Xu, M. Qi, S. Wen, Y. Xiang, and G. Nan, "A survey of ethereum smart contract security: attacks and detection," *Distributed Ledger Technologies: Research and Practice*, vol. 3, no. 3, pp. 1–28, 2024.

[8] A. Mense and M. Flatscher, "Security vulnerabilities in ethereum smart contracts," in *Proceedings of the 20th international conference on information integration and web-based applications & services*, 2018, pp. 375–380.

[9] A. Singh, R. M. Parizi, Q. Zhang, K.-K. R. Choo, and A. Dehghantanha, "Blockchain smart contracts formalization: Approaches and challenges to address vulnerabilities," *Computers Security*, vol. 88, p. 101654, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404818310927

[10] M. Ren, Z. Yin, F. Ma, Z. Xu, Y. Jiang, C. Sun, H. Li, and Y. Cai, "Empirical evaluation of smart contract testing: What is the best choice?" in *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, 2021, pp. 566–579.

[11] C. Chen, J. Su, J. Chen, Y. Wang, T. Bi, J. Yu, Y. Wang, X. Lin, T. Chen, and Z. Zheng, "When chatgpt meets smart contract vulnerability detection: How far are we?" *ACM Transactions on Software Engineering and Methodology*, 2023.

[12] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International conference on software engineering*, 2020, pp. 530–541.

[13] S. Hwang and S. Ryu, "Gap between theory and practice: An empirical study of security patches in solidity," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 542–553.

[14] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on ethereum smart contracts (sok)," in *Principles of Security and Trust: 6th International Conference, POST 2017*, 2017, pp. 164–186.

[15] K. Li, Y. Xue, S. Chen, H. Liu, K. Sun, M. Hu, H. Wang, Y. Liu, and Y. Chen, "Static application security testing (sast) tools for smart contracts: How far are we?" *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1447–1470, 2024.

[16] H. Chen, M. Pendleton, L. Njilla, and S. Xu, "A survey on ethereum systems security: Vulnerabilities, attacks, and defenses," *ACM Computing Surveys (CSUR)*, vol. 53, no. 3, pp. 1–43, 2020.

[17] Z. A. Khan and A. S. Namin, "A survey of vulnerability detection techniques by smart contract tools," *IEEE Access*, 2024.

[18] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu, "Large language model-powered smart contract vulnerability detection: New perspectives," in *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. IEEE, 2023, pp. 297–306.

[19] P. Ince, J. Yu, J. K. Liu, and X. Du, "Generative large language model usage in smart contract vulnerability detection," *arXiv preprint arXiv:2504.04685*, 2025.

[20] D. Wong and M. Hemmel, "Decentralized application security project top 10 of 2018," 2018.

[21] M. Di Angelo and G. Salzer, "A survey of tools for analyzing ethereum smart contracts," in *2019 IEEE international conference on decentralized applications and infrastructures (DAPPCON)*, 2019, pp. 69–78.

[22] S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, and H.-N. Lee, "Ethereum smart contract analysis tools: A systematic review," *Ieee Access*, vol. 10, pp. 57 037–57 062, 2022.

[23] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.

[24] (2018) Consensys/mythril. [Online]. Available: https://github.com/Consensys/mythril

[25] "Cyfrin aderyn: Smart contract vulnerability detection tool," 2024. [Online]. Available: https://github.com/Cyfrin/aderyn?tab=readme-ov-file

[26] I. Nikolic, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, "Finding the greedy, prodigal, and suicidal contracts at scale," *CoRR*, vol. abs/1802.06038, 2018. [Online]. Available: http://arxiv.org/abs/1802.06038

[27] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2016, pp. 254–269. [Online]. Available: https://doi.org/10.1145/2976749.2978309

[28] C. F. Torres, M. Steichen, and R. State, "The art of the scam: Demystifying honeypots in ethereum smart contracts," in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1591–1607. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/ferreira

[29] C. F. Torres, J. Schütte, and R. State, "Osiris: Hunting for integer bugs in ethereum smart contracts," in *Proceedings of the 34th Annual Computer Security Applications Conference*. Association for Computing Machinery, 2018, pp. 664–676. [Online]. Available: https://doi.org/10.1145/3274694.3274737

[30] Z. Zheng, N. Zhang, J. Su, Z. Zhong, M. Ye, and J. Chen, "Turn the rudder: A beacon of reentrancy detection for smart contracts on ethereum," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 295–306.

[31] Ethereum block explorer. [Online]. Available: https://etherscan.io/

[32] Z. Wang, J. Chen, Y. Wang, Y. Zhang, W. Zhang, and Z. Zheng, "Efficiently detecting reentrancy vulnerabilities in complex smart contracts," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 161–181, 2024.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on May 04,2026 at 07:37:33 UTC from IEEE Xplore. Restrictions apply.